

Implementation of AND gate

Introduction:

The AND gate is a combinational logic gate that performs the logical AND operation. Its output is a logic high (1) only if all its inputs are high (1); otherwise, the output is low (0).

For a 2-input AND gate, the Boolean expression is:

$$Y = A \cdot B$$

Where:

A, B are binary input signals

Y is the binary output

Block diagram:

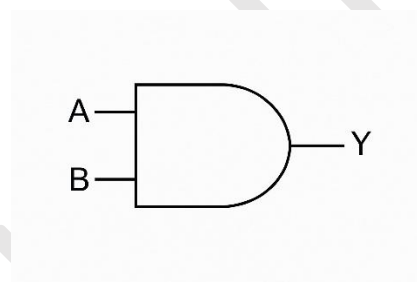


Fig: Block Diagram of AND Gate

Truth table:

A	B	Y = A & B
0	0	0
0	1	0
1	0	0
1	1	1

Application:

AND gates are used in:

- Arithmetic circuits (like adders)
- Control circuits (decision making)
- Enable/disable logic
- Multiplexers, decoders, and more

They form the backbone of combinational logic circuits, which are essential in designing microprocessors, FPGAs, ASICs, and embedded systems.

Procedure to create project:

Step 1: Set up Project Directory

- 1.1: Create a directory for your lab: `mkdir <project_name>`
- 1.2: Open lab directory: `cd <Project_name>`
- 1.3: Create directory for project: `mkdir and_gate`
- 1.4: Open project directory: `cd and_gate`

Step 2: Create Design file

- 2.1: Open the gvim editor for design with command `gvim and_gate_design.v`

RTL Design Code:

```
module and_gate_design (  
    input A,  
    input B,  
    output out  
);
```

```
assign out = A&B;  
endmodule
```

- 2.2: Save the file using command- `:wq!`

Step 3: Create Testbench File

- 3.1: Open the gvim editor for testbench with command: `gvim and_gate_tb.v`

Testbench:

```
module and_gate_tb;  
    reg A;  
    reg B;  
    wire out;  
    and_gate_design uut(.A(A), .B(B), .out(out));  
    initial begin  
        $display (" time=%t A = %0b| B =%0b |out =%b", $time, A, B, out);
```

```
$monitor (" time=%t A = %0b| B =%0b|out = %b", $time,A,B,out);  
#10;  
    A = 1'b0; B = 1'b0 ; #10;  
    A = 1'b0; B = 1'b1 ; #10;  
  
    A = 1'b1; B = 1'b0 ; #10;  
    A = 1'b1; B = 1'b1 ; #10;  
$finish;  
end  
initial begin  
    $dumpfile("and_gate_dump.vcd");  
    $dumpvars();  
end  
endmodule
```

3.2: Save the testbench file using command :wq!

3.3: Check the saved file in current directory: use command "ls"

Step 4: : Simulate the Design with Verilator

4.1: Use the following command to simulate

```
verilator --binary -j 0 -Wall and_gate_design.v \  
and_gate_tb.v --top and_gate_tb --timing --CFLAGS \  
"-std=c++20" --trace
```

4.2: If any error is available, resolve it.

4.3: After simulation a new directory obj_dir will be created by tool, check it by using command "ls".

4.4: Open obj_dir

4.5: Use command make -f Vand_gate_tb.mk Vand_gate_tb

4.6: A new directory with name Vand_gate_tb will be created

4.7: Use command ./Vand_gate_tb to compile

4.8: Check output at terminal

Simulation output:

```

time=          0 A = 0 | B =0 | out =0
time=          0 A = 0 | B =0 | out = 0
time=         20 A = 0 | B =1 | out = 0
time=         30 A = 1 | B =0 | out = 0
time=         40 A = 1 | B =1 | out = 1
- and_gate_tb.sv:20: Verilog $finish
    
```

The output shown is the simulation result of an AND gate implemented in Verilog. At the beginning of the simulation the signals are reset to binary 0. After this with the proceeding each simulation time step, the inputs A and B are changed, and the output of the AND gate is displayed. From the waveform the expected behaviour of the AND gate can be verified. The AND gate generates output 1 only when both inputs A and B are 1, otherwise the output remains 0. The test concludes with the \$finish system task at time 40.

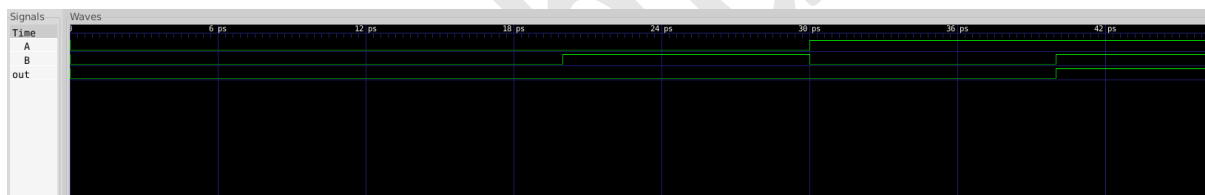
Step 5: View Waveform Output

5.1: Launch GTKWave to view the waveform file:

```
gtkwave dump_and_gate.vcd
```

5.2: Analyze the waveform to verify the correct operation of the 4-bit up counter

Output Waveform:



The above waveform shows the timing diagram of a 2-input AND gate simulation, which aligns with the testbench results discussed earlier. The X-axis represents simulation time, measured in time units. The Y-axis shows the logic level of each signal:

- Low (0) is represented by the lower level of a waveform
- High (1) is represented by the upper level of a waveform

This confirms that the AND design is functioning correctly.